

APAI2025 - LAB05

Tiny Transformers, Tiny Stories

Authors: Luca Bompani, Davide Nadalini, Alberto Dequino, Francesco Conti

Contacts: d.nadalini@unibo.it

Links: [GitHub Link \(code\)](#)

Summary

1. Subject(s):
 - Attention on the PULP architecture
 - LLama Transformer
 - profiling code execution
2. Programming Language: C
3. Lab duration: 3h
4. 4. Assignment:
 - Time for delivery: 2 weeks
 - **Submission deadline: Nov 7st 2025**

How to deliver the assignment

You will deliver ONLY THIS TEXT FILE, no code

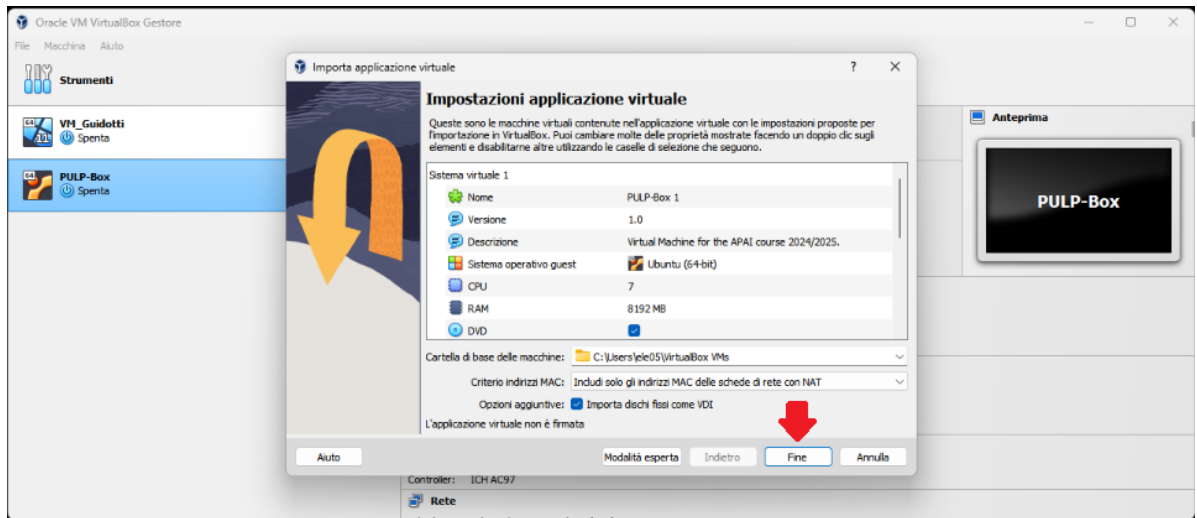
- Download this file.
- Fill in the required results.
- Export to pdf format.
- Rename the file to: LAB<number_of_the_lesson>_APAI_<your_name>.pdf
- Use Virtuale platform to load ONLY your .pdf file

LAB STARTS HERE

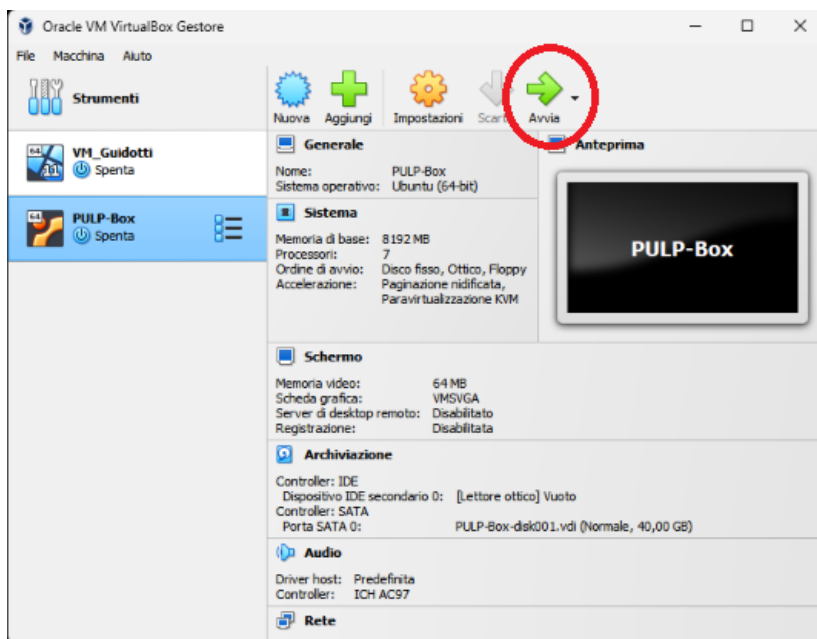
0. (ONLY LAB1) Access to the local VM

- On the lab's PCs, open the file explorer and go to This PC, C:/VM_APAI

- Double click on PULP-box.ova
- VirtualBox opens, just click on “Fine”



- Wait for the VM to be imported
- Open the VM with “Avvia”

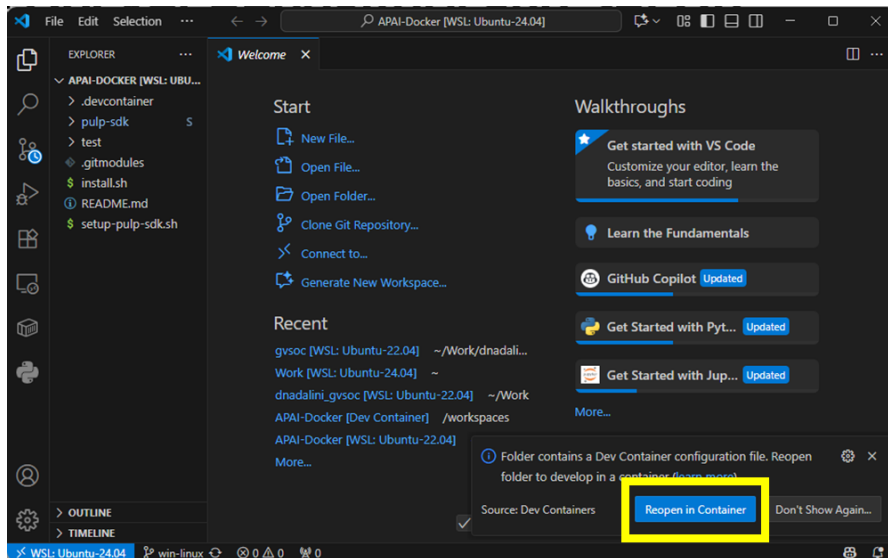


- Password is ‘pulp’

1. (ALL) Open Docker Container with VSCode

- Open a terminal (right click – open a new terminal)
- Go in the APAl-Docker folder: `cd APAl-Docker`

- Open Docker with VSCode: `code .`
- Select the pop-up option “Reopen in container”

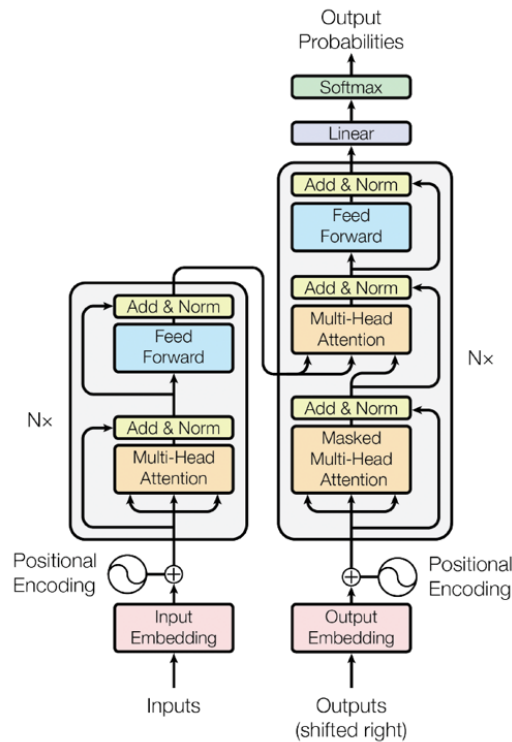


- Setup PULP-SDK: `source setup-pulp-sdk.sh`
- Clone GitHub repository of today's lab: `git clone https://github.com/EEESlab/APAI25-LAB05-TinyTransformers`
- `cd APAI25-LAB05-TinyTransformers`
- `source initialization.sh`
- `make clean all run`

Introduction: LLama2 in tiny pills

Before starting, be sure to “source initialization.sh” in today's exercise folder.

Starting from the Transformer architecture:

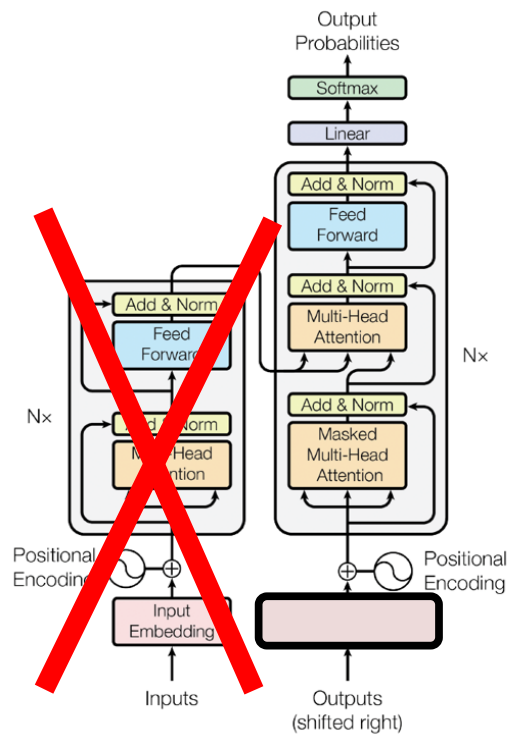


It is commonly divided into two parts: the encoder, on the left, and the decoder, on the right.

This structure was the original one proposed in [Attention is All You Need](#).

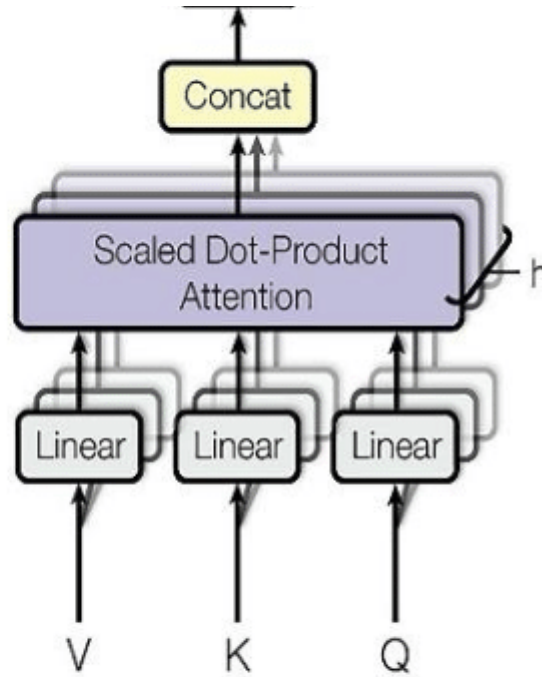
For text generation, however, another architecture is more commonly used.

It is called decoder-only and unifies the encoder with the first layers of the decoder, using it for both input encoding and output auto-regression.



The Transformer we are seeing today is called LLama and it is based on this second approach.

Task 1: Attention



LLAMA2, as all transformers, is based on the attention mechanism. Such a mechanism is based on three tensors, computed from the input data, which are the Value (V), the Key (K) and the Query (Q). These tensors are divided between multiple heads, which provide attention to only a subset of the input features. In particular, LLAMA2 uses grouped query attention, which shares K and V among multiple heads. This means that, given N^H attention heads, if every head will feature a different Q tensor, there will be groups of heads sharing the same K and V. The number of different groups of K and V tensors is, then, N^{KV} , which are key-value heads. Instead, every token in the input sequence, which will comprise s elements (i.e., the input which is then transformed into Q, K and V) will feature a size equal to dim . The weights related to Q, K and V, which project the input embedding into the attention block, feature a size of:

$$W^Q = dim * dim * \frac{N^H}{N^H}$$

$$W^K = dim * dim * \frac{N^{KV}}{N^H}$$

$$W^V = dim * dim * \frac{N^{KV}}{N^H}$$

Calculate the size of each of these tensors using the formula above (the data values can be found in the `conf_and_weights_fp16.h` header file):

$$W^Q = 64 * 64 * \frac{8}{8} = 4096$$

$$W^K = 64 * 64 * \frac{4}{8} = 2048$$

$$W^V = 64 * 64 * \frac{4}{8} = 2048$$

After computing the size of these weights, let's implement their computation. In the provided code, substitute the missing parts in the forward function. These weights are stored in the **TransformerWeights** data structure. In order to execute the attention block, the data should be moved into the Cluster memory, called the L1. To do so, we will use the Cluster DMA, which can move the data between the system level memory (L2) and the Cluster memory (L1). Complete the code in the **net.c** file to move the weights for the Q, K and V projections in the L1 memory. Note that the weight of K and V are stored in the same matrix, while Q's ones are separate.

```
/**
 * TASK 1:
 *
 * Complete this function. Be aware: the execution of the matmul is down in the code, not here.
 * memcpy_setup is define in this file: look inside the function to understand the arguments.
 */
memcpy_setup(____,PI_CL_DMA_DIR_EXT2LOC,&kv_weight);
```

```
/**
 * TASK 1:
 *
 * Complete this function. Be aware: the execution of the matmul is down in the code, not here.
 * memcpy_setup is define in this file: look inside the function to understand the arguments.
 */
memcpy_setup((w->wq + l*dim*dim),____,dim*dim*sizeof(*w->wq), PI_CL_DMA_DIR_EXT2LOC,&q_weight);
```

```
/**
 * TASK 1:
 *
 * Set the size of the transfer. The dimensions of the KV weights are l, dim, kv_dim.
 */
kv_weight.ext = (uint32_t) (w->wk + /*Transfer size*/);
pi_cl_dma_memcpy(&kv_weight);
```

Put a screenshot or copy your code here:

```
/**
 * TASK 1:
 *
 * Complete this function. Be aware: the execution of the matmul is down in the code, not here.
 * memcpy_setup is define in this file: look inside the function to understand the arguments.
 */
memcpy_setup((w->wv + l*dim*kv_dim), BUFF_W_2, dim*kv_dim*sizeof(*w->wv), PI_CL_DMA_DIR_EXT2LOC, &kv_weight);
```

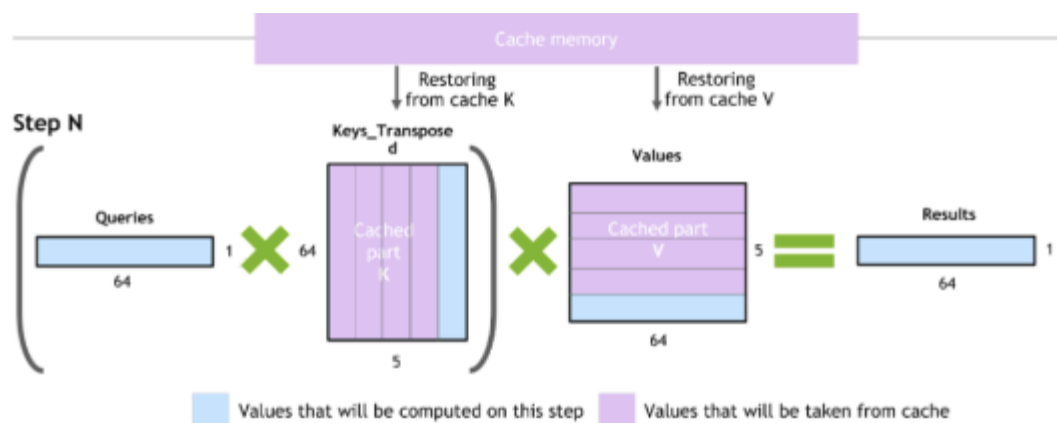
```
/**
 * TASK 1:
 *
 * Complete this function. Be aware: the execution of the matmul is down in the code, not here.
 * memcpy_setup is define in this file: look inside the function to understand the arguments.
 */
memcpy_setup((w->wq + l*dim*dim), BUFF_W_1, dim*dim*sizeof(*w->wq), PI_CL_DMA_DIR_EXT2LOC, &q_weight);
```

```
/**
 * TASK 1:
 *
 * Set the size of the transfer. The dimensions of the KV weights are l, dim, kv_dim.
 */
kv_weight.ext = (uint32_t) (w->wk + l*dim*kv_dim);
```

After moving the weights in the L1 memory, the code will then compute their multiplication with the input embedding, obtaining the K V, and Q tensors.

You may have noticed a so-called “KV cache”. This is an optimization to reduce the number of computations during the autoregressive phase. During this phase, the output of TinyLlama is a function of both the input embedding and the previous outputs of the model. During the iterative autoregressive phase, which is the one in which the model generates text related to the input prompt, the model will start generating a single output token per iteration, based on the input and the previously generated outputs. The generation process will start when the input sequence is provided to the model (generating a starting token) and will progress until the model produces a stop token.

During every iteration of this phase, without optimizations, the model should recompute the K and V tensors and the attention matrix based on all the previous tokens in the sequence (i.e., the input and the previous outputs). However, this increases the computation costs a lot. Therefore, the KV cache is a matrix that stores a pre-defined number of previously computed K and V values. The KV cache acts as a shift register, where the oldest K and V values are discarded, keeping the memory cost bounded. Its behaviour is depicted in the image below.



We now have all the elements to perform the attention, which is composed of a series of matrix vector operations. We will now implement missing code concerning the attention block (the Multi-Head Self-Attention or MHSA).

Complete the code in the **MHSA.c** file:

```
v2f16 temp1 = (v2f16) {0, 0};
v2f16 temp2 = (v2f16) {0, 0};
v2f16 temp3 = (v2f16) {0, 0};
v2f16 temp4 = (v2f16) {0, 0};
v2f16 A, B1, B2, B3, B4;
for (int i = 0; i < head_size; i+=2) { //+=2 because each vector is composed of 2 elmenets
    A = *((v2f16*) &q[i]);
    B1 = *((v2f16*) &k[i]);
    B2 = *((v2f16*) &k[i + kv_dim]);
    B3 = *((v2f16*) &k[i + 2*kv_dim]);
    B4 = *((v2f16*) &k[i + 3*kv_dim]);
    /**
     * TASK 1:
     *
     * Complete the code with the scalar product between the Query Q and the Key K, among the embedding dimensions.
     * We already extracted the values from the buffers: what do you need to do here?
     * Remember you have multiple keys for each query.
     */
}
```

```
v2f16 temp = (v2f16) {0, 0};
v2f16 A,B;
for (int i = 0; i < head_size; i+=2) {
    A = *((v2f16*) &q[i]);
    B = *((v2f16*) &k[i]);
    /**
     * TASK 1:
     *
     * Complete the code with the scalar product along the embedding dimensions.
     * We already extracted the values from the buffers: what do you need to do here?
     */
}
```

Put a screenshot or copy your code here:

```
v2f16 temp1 = (v2f16) {0, 0};
v2f16 temp2 = (v2f16) {0, 0};
v2f16 temp3 = (v2f16) {0, 0};
v2f16 temp4 = (v2f16) {0, 0};
v2f16 A, B1, B2, B3, B4;
for (int i = 0; i < head_size; i+=2) { //+=2 because each vector is composed of 2 elmenets
    A = *((v2f16*) &q[i]);
    B1 = *((v2f16*) &k[i]);
    B2 = *((v2f16*) &k[i + kv_dim]);
    B3 = *((v2f16*) &k[i + 2*kv_dim]);
    B4 = *((v2f16*) &k[i + 3*kv_dim]);
    /**
     * TASK 1:
     *
     * Complete the code with the scalar product between the Query Q and the Key K, among the embedding dimensions.
     * We already extracted the values from the buffers: what do you need to do here?
     * Remember you have multiple keys for each query.
     */
    temp1 += A * B1;
    temp2 += A * B2;
    temp3 += A * B3;
    temp4 += A * B4;
}
```

```
v2f16 temp = (v2f16) {0, 0};
v2f16 A,B;
for (int i = 0; i < head_size; i+=2) {
    A = *((v2f16*) &q[i]);
    B = *((v2f16*) &k[i]);
    /**
     * TASK 1:
     *
     * Complete the code with the scalar product along the embedding dimensions.
     * We already extracted the values from the buffers: what do you need to do here?
     */
    temp += A * B;
}
```

Task 2: Using your transformer

Now, let's go to TinyLlama's execution. In every iteration, the transformer will, at first, do a forward step to compute the prediction with respect to its inputs and the previous tokens (stored in the KV cache). Then, the model will sample the correct token among the outputs of the current inference. Finally, it will do the decoding. Following the scheme of the transformer architecture, complete the code in the "net_step()" function of **net.c** file. Try to read their source code to gain more insights of the inner mechanisms on how the model actually works.

To complete this exercise, simply call the correct function for each location.

First, uncomment this code:

```
/**
 * TASK 2:
 *
 * Complete the code of the autoregressive phase with the missing steps.
 * UNCOMMENT THE CODE BELOW, FIRST!!
 */

//Missing function with these arguments: (&tokenizer, PROMPT, 1, 0, prompt_tokens, &num_prompt_tokens);

// token = prompt_tokens[0];
// for(int pos = 0; pos < steps; pos++ ) {

//     log = 0; //Missing function with these arguments: (&transformer, token, pos);

//     if(pos < num_prompt_tokens -1)
//         next = prompt_tokens[pos+1];
//     else{
//         next = 0; //Missing function with these arguments: (&sampler, log, pos==STEPS-1);
//     }

//     if(next==1)
//         break;

//     char* piece = 0; //Missing function with these arguments: (&tokenizer, token, next);

//     safe_printf(piece);

//     token = next;
// }
```

Then, complete it with the missing functions:

```

/**
 * TASK 2:
 *
 * Complete the code of the autoregressive phase with the missing steps.
 * UNCOMMENT THE CODE BELOW, FIRST!!
 */

//Missing function with these arguments: (&tokenizer, PROMPT, 1, 0, prompt_tokens, &num_prompt_tokens);

token = prompt_tokens[0];
for(int pos = 0; pos < steps; pos++ ) {

    log = 0; //Missing function with these arguments: (&transformer, token, pos);

    if(pos < num_prompt_tokens -1)
        next = prompt_tokens[pos+1];
    else{
        next = 0; //Missing function with these arguments: (&sampler, log, pos==STEPS-1);
    }

    if(next==1)
        break;

    char* piece = 0; //Missing function with these arguments: (&tokenizer, token, next);

    safe_printf(piece);

    token = next;
}

```

All the functions are already declared and defined in the various .c files included in this laboratory.

Put a screenshot or copy your code here:

```
/**
 * TASK 2:
 *
 * Complete the code of the autoregressive phase with the missing steps.
 * UNCOMMENT THE CODE BELOW, FIRST!!
 */

encode(&tokenizer, PROMPT, 1, 0, prompt_tokens, &num_prompt_tokens); // from Tokenizer.c

token = prompt_tokens[0];
for(int pos = 0; pos < steps; pos++ ) {

    log = forward(&transformer, token, pos); // from net.c

    if(pos < num_prompt_tokens -1)
    |   next = prompt_tokens[pos+1];
    else{
    |   next = sample(&sampler, log, pos==STEPS-1); // from Sampler.c
    }

    if(next==1)
    |   break;

    char* piece = decode(&tokenizer, token, next); // from Tokenizer.c

    safe_printf(piece);

    token = next;
}
```

After completing the task, activate profiling for layer $l == 0$ using the `RESET_STATS()`, `START_STATS()` and `STOP_STATS()` macros defined in the `stats.h` header.

You must also enable them by uncommenting the following line in the main folder's Makefile:

```
# APP_CFLAGS += -DSTATS
```

Answer the following questions:

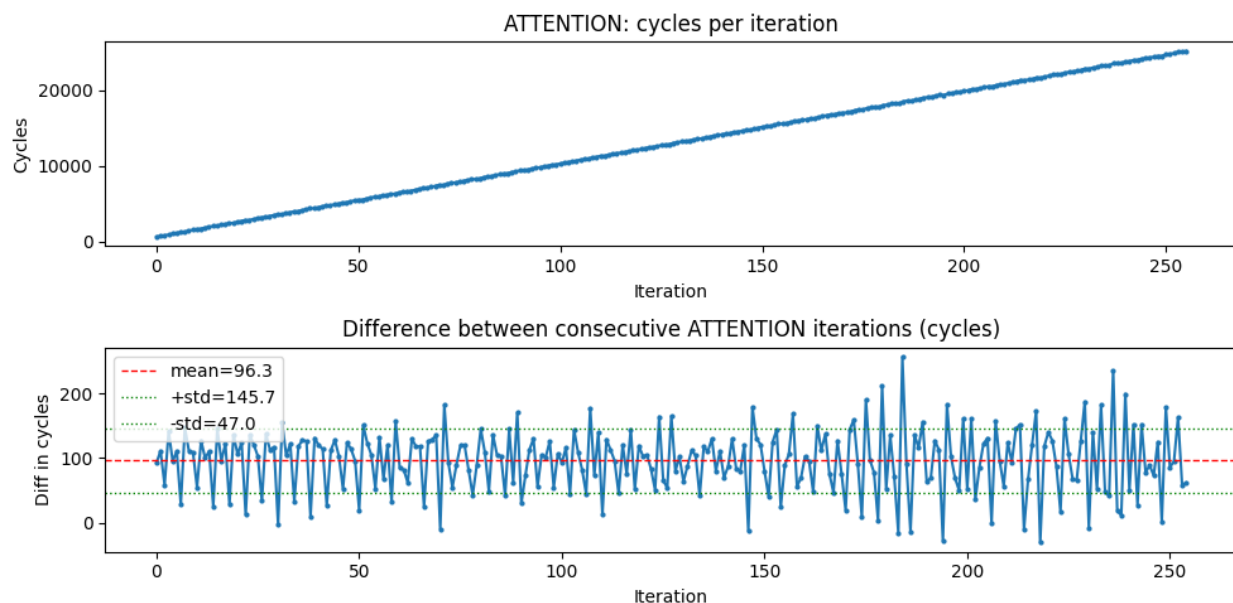
What is the number of cycles to compute the K, Q, and V tensors? You can see there are many of them (autoregressive mode), please report one of the values.

- V (matmul): 724 cycles
- Q (matmul): 1070 cycles
- K (matmul): 716 cycles

Q requires more cycles than V and K because the latter two correspond to smaller matrix multiplications of size $dim * kv_dim$, whereas Q involves a larger $dim * dim$ operation. This difference in computational load is also evident in the instruction count ($V \approx K = 476$, $Q = 806$) and active cycles ($V \approx K = 680$, $Q = 1016$).

What is the number of cycles necessary for the calculation of the attention? If you look at the numbers of every iteration, do you notice a change in their value?

To answer this question i extracted the output and plotted the values of the Attention block and noticed that the cycles increased constantly from 627 to 25184. As we can see they increase relatively slowly and each increase between consecutives iterations oscillates between 0 and 200 cycles.



In autoregressive transformers, each new token attends to all previous ones. Even with KV caching, the query still computes dot products with all stored keys, so cycles grow roughly linearly with sequence length. The small per-step increases (0–200 cycles)

reflect one extra key/value pair per step. Minor oscillations come from hardware effects like memory contention or cache alignment.

Answers must be commented on.

Try to give an explanation of the obtained results.

Task 3: Run the model

With the previous steps, we then completed the code of our TinyLlama. In this step, we will generate a story with our transformer! To do so, run “make clean all run”.

Run the full model and copy the story that has been generated:

Once upon a time, there was a brave bird named Spot. Spot loved to run and play in the sky. One day, Spot found a curch in his birthday grandma. He was very angry and white.

The bird saw a big cat named Max. Max was sad. He asked his friend, a little morning, "What do you see?" Max said, "I'm a monster! I found another cushion toy!" The cat said, "I want to see it!" Max said, "We can have the cacture and go on the chicken." Max was so happy and flew down the cushion again.